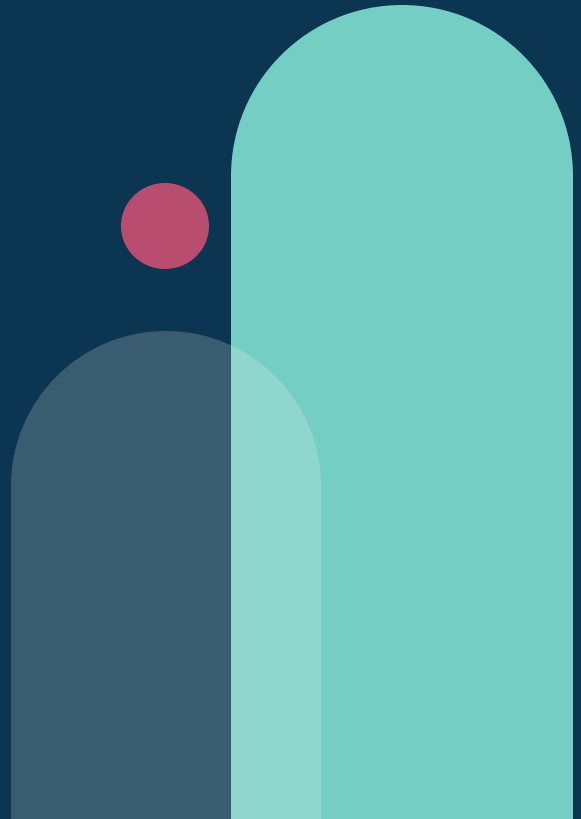
Basics of AI/ML/DL

Vectors

01

Vectors



Introduction to Vectors

What are Vectors?

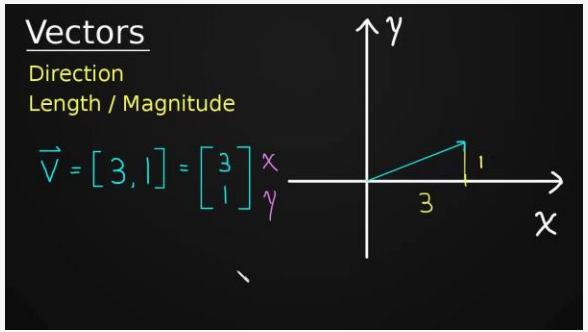
Vectors are mathematical representations of data points in a continuous space, often used to represent features or attributes in machine learning.

Why Vectors?

Vectors enable mathematical operations like addition, scaling, and dot products, which are fundamental to many machine learning algorithms.

Applications:

Used in tasks like document similarity, image processing, and recommendation systems.



https://www.desmos.com/calculator/u8wt5rnw9n
https://www.math3d.org/obnDo2J1w

Embeddings

What are Embeddings?

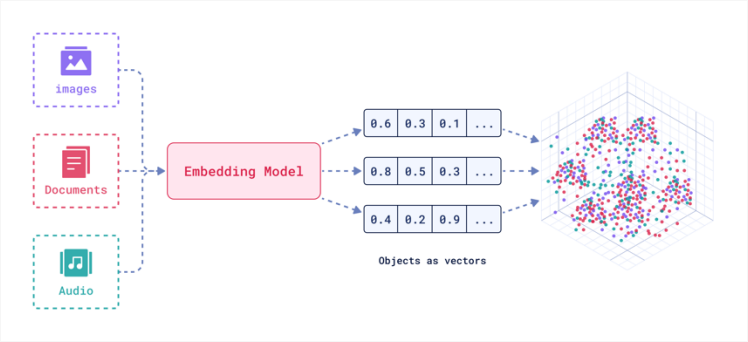
Embeddings are dense vector representations of data, particularly high-dimensional categorical data like words.

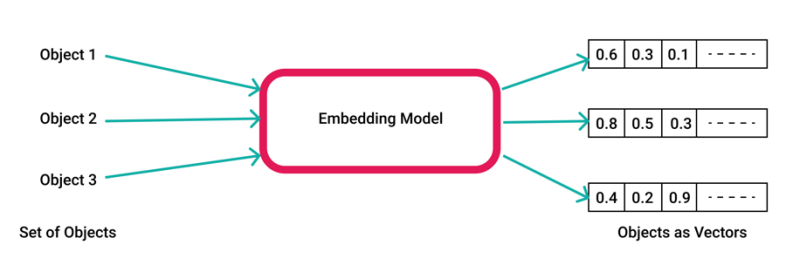
Purpose:

Reduce the dimensionality of data while preserving its semantic relationships.

Types of Embeddings:

Word Embeddings: Represent words in a continuous vector space, capturing their meanings.





Creating Vectors in NumPy

Code Example:

```
import numpy as np

# Creating simple vectors
vector1 = np.array([1, 2, 3])
vector2 = np.array([4, 5, 6])

print('Vector 1:', vector1)
print('Vector 2:', vector2)
```

Visualizing Vectors

Vectors can be thought of as points or arrows in a coordinate system.

Code Example:

```
import matplotlib.pyplot as plt

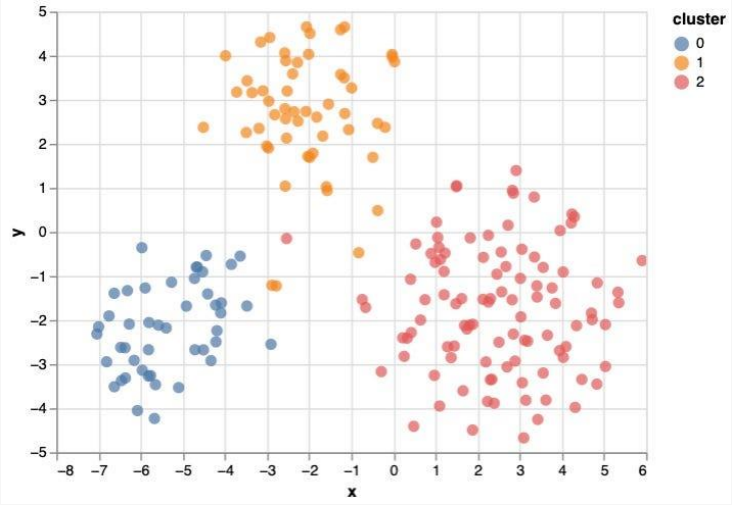
plt.quiver(0, 0, vector1[0], vector1[1], angles='xy', scale_units='xy', scale=1, color='r')
plt.quiver(0, 0, vector2[0], vector2[1], angles='xy', scale_units='xy', scale=1, color='b')
plt.xlim(0, 7)
plt.ylim(0, 7)
plt.show()
```

Measuring Vector Similarity

Core Concept:

Vector similarity measures how similar two vectors are in the vector space.

Vector similarity is like comparing friends' movie tastes or songs on a playlist. If preferences or sounds are alike, they're close together on a map, showing how similar they are.



Cosine Similarity

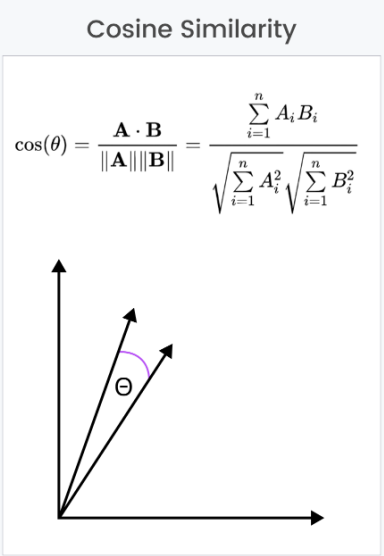
Introduction to Cosine Similarity:

Cosine similarity is a common metric that measures the cosine of the angle between two vectors

```
def cosine_similarity(v1, v2):
    dot_product = np.dot(v1, v2)
    norm_v1 = np.linalg.norm(v1)
    norm_v2 = np.linalg.norm(v2)
    return dot_product / (norm_v1 * norm_v2)

similarity = cosine_similarity(vector1, vector2)
print('Cosine Similarity:', similarity)
```

Values closer to 1 indicate high similarity. They range from -1 to 1



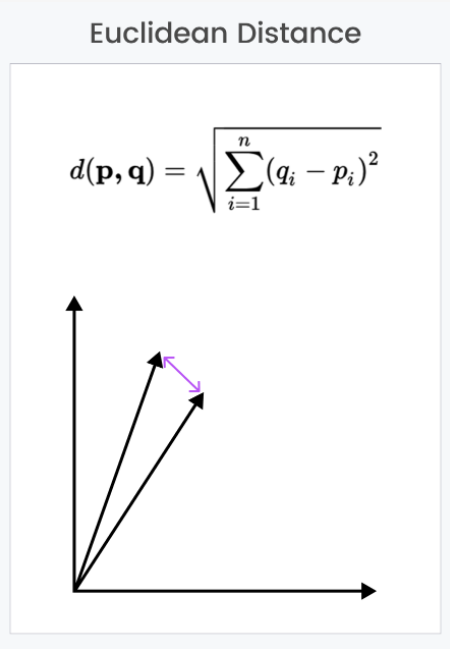
Euclidean Distance

Introduction: Another method to measure similarity is by calculating the Euclidean distance, which measures the 'straight line' distance between two vectors.

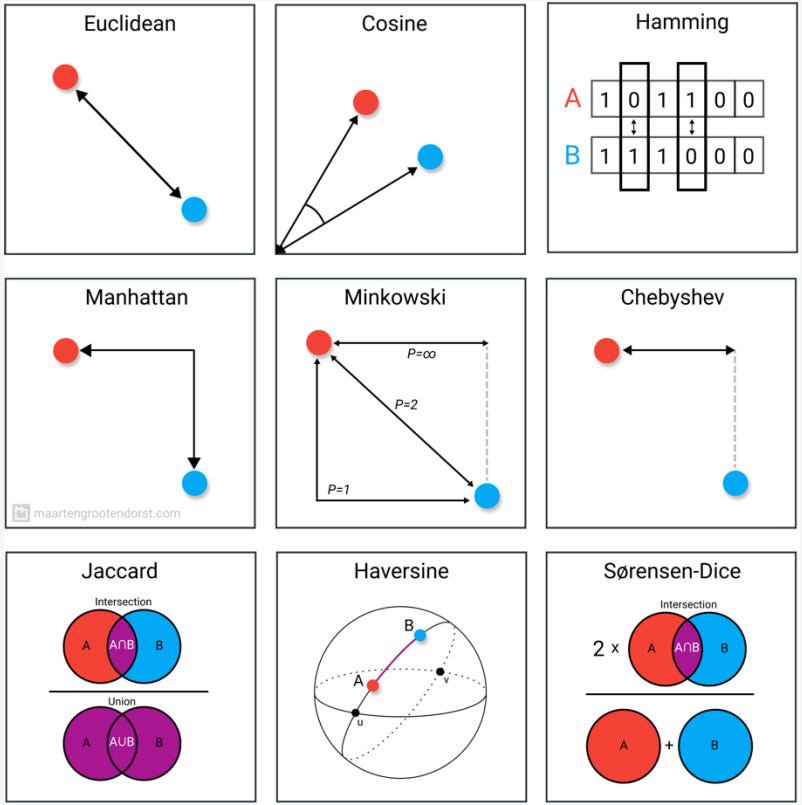
Code Example:

```
def euclidean_distance(v1, v2):
    return np.linalg.norm(v1 - v2)

distance = euclidean_distance(vector1, vector2)
print('Euclidean Distance:', distance)
```



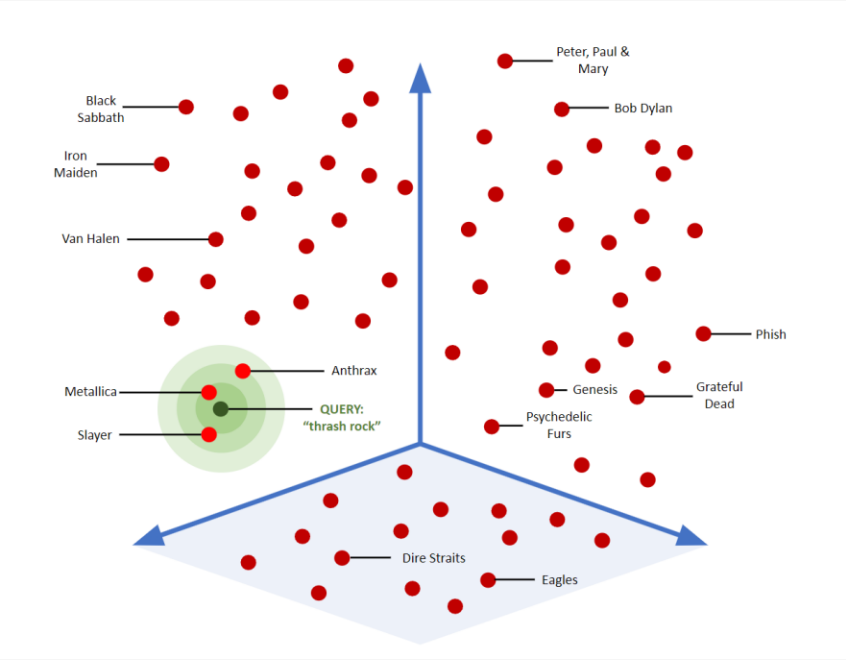
Other Metrics



Introduction to Vector Search Algorithms

Concept: Vector search algorithms are designed to efficiently find the nearest neighbors in high-dimensional vector spaces.

Why It's Important: Efficient vector search is critical in applications where real-time performance is necessary, such as recommendation systems, image retrieval, and natural language processing.

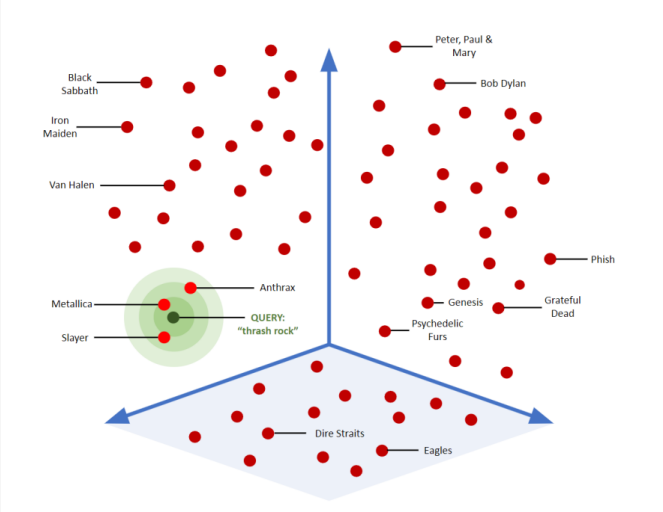


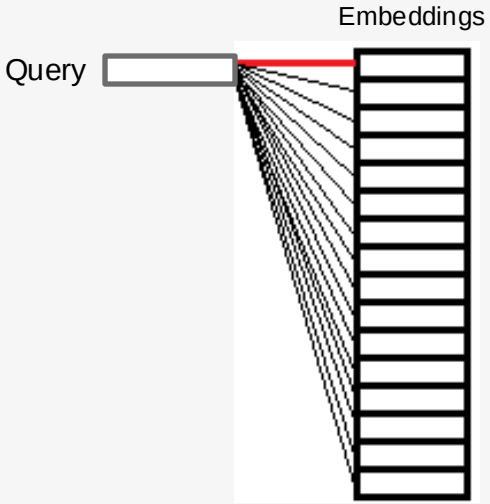
Key Vector Search Algorithms

Brute Force Search:
How it works: Compares the query vector to every other vector in the dataset using a similarity measure (e.g., cosine similarity).
Pros: Simple and exact.
Cons: Computationally expensive, especially with large datasets.

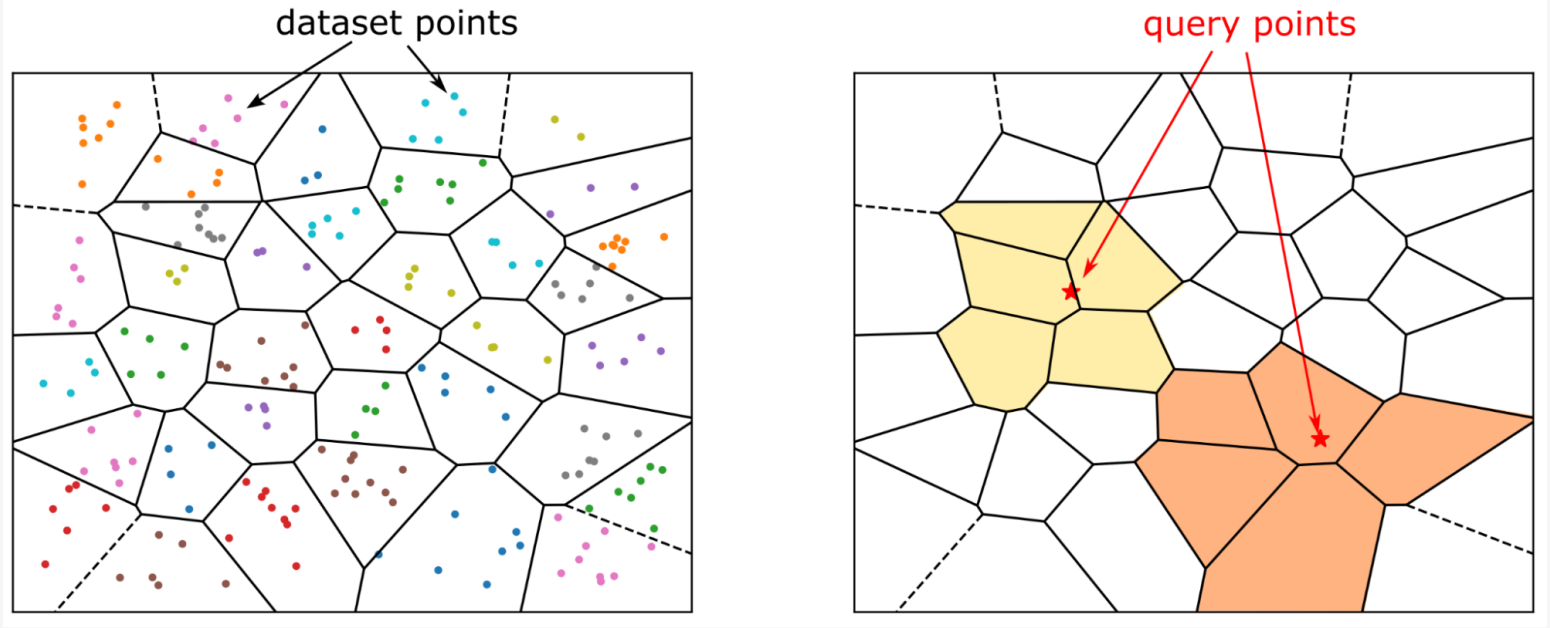
Approximate Nearest Neighbor (ANN):
How it works: Finds approximate nearest neighbors using techniques like HNSW and LSH.
Pros: Significantly faster than brute force, especially for large datasets.
Cons: May sacrifice some accuracy for speed.

Bruteforce

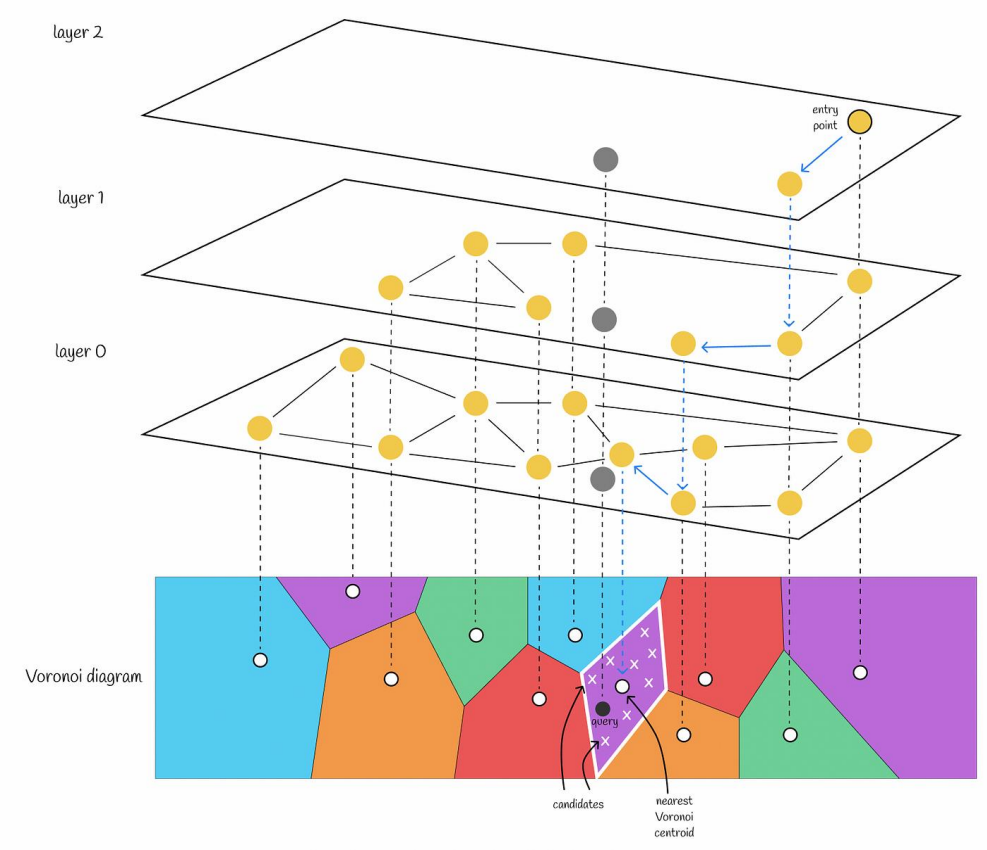

Peter, Paul & Mary
Black Sabbath
Bob Dylan
Iron Maiden
Van Halen
Phish
Anthrax
Metallica
QUERY: "thrash rock"
Genesis
Grateful Dead
Slayer
Psychedelic Furs
Dire Straits
Eagles


Embeddings
Query

IVFFlat



HNSW


layer 2
entry point
layer 1
layer 0
Voronoi diagram
query
candidates
nearest Voronoi centroid

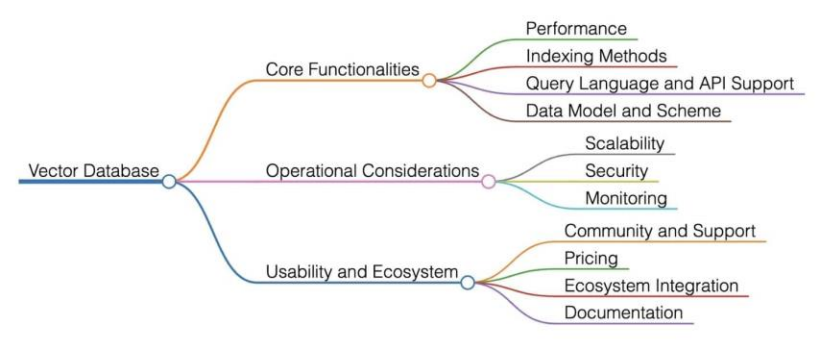
The Need for Vector Databases

Problem Statement:

As datasets grow larger, it becomes computationally expensive to search for similar vectors using brute force methods.

Solution:

Vector Databases (VecDB) are designed to efficiently **store, index, and retrieve high-dimensional vectors.**



What is a Vector Database?

Definition:

A vector database is a specialized database optimized for handling and querying large sets of high-dimensional vectors.

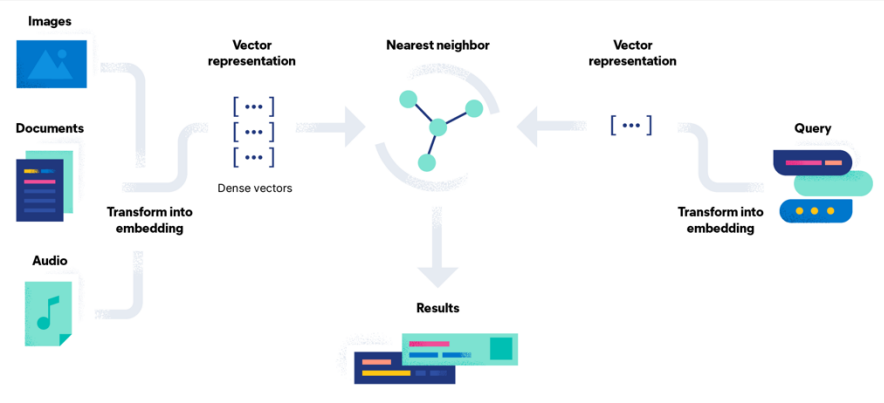
Why It's Important:

Essential for scaling applications that rely on **vector search,** such as recommendation systems, image search, and NLP tasks.

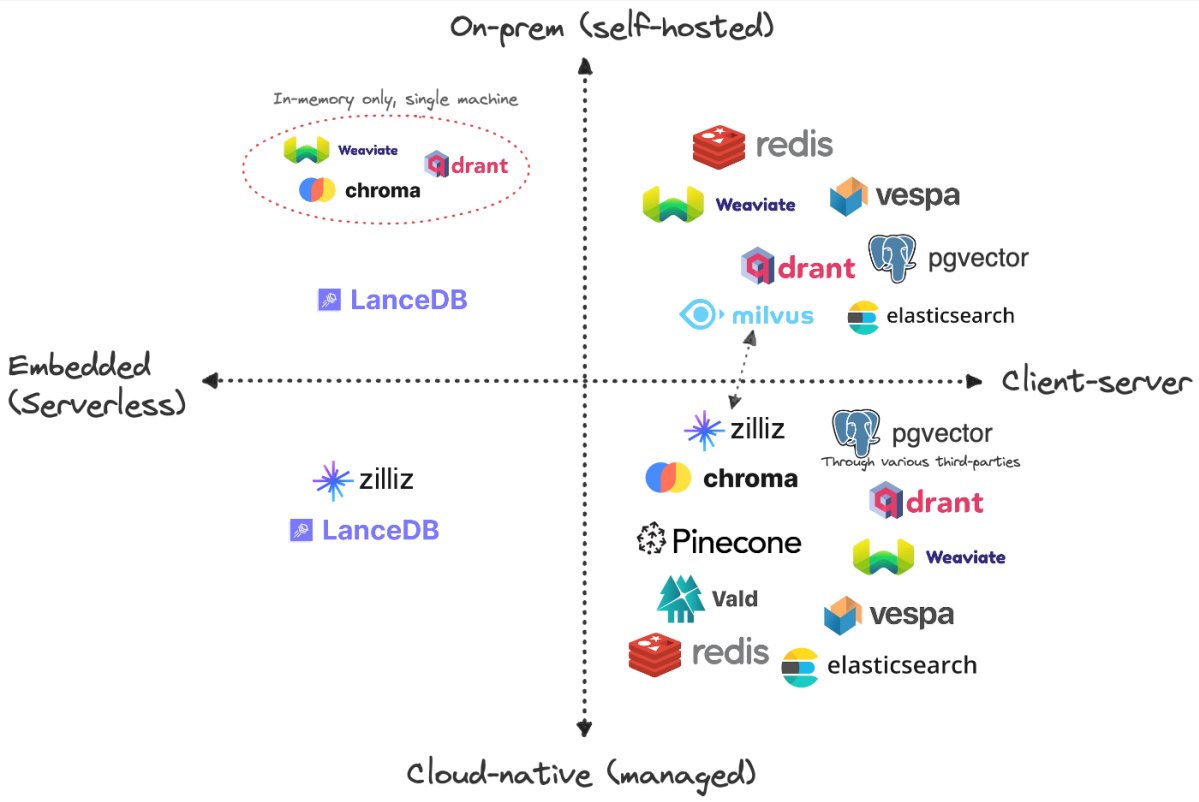
How Vector Databases Work

Core Components:

1. **Storage Engine:** Efficiently stores large amounts of vector data.
2. **Indexing:** Utilizes techniques like HNSW or IVF to speed up search.
3. **Query Engine:** Processes and returns results based on similarity or distance metrics.



Popular Vector Databases



Hands-On Example

Thanks!

Do you have any questions?

